

Initiation à Qt

Création d'interfaces graphiques

Découverte des bases de Qt en 2,5 heures



Yassine NASSER
Polytech'Orléans – site Galilée, Orléans F 212

Cours d'initiation à Qt

1. Introduction à Qt et configuration (30 minutes)

- **Objectif** : Comprendre l'écosystème Qt et configurer l'environnement de développement.
- **Contenu** :
 - Présentation de Qt, Installation, Première application GUI (Graphical User Interface)

2. Bases de l'interface graphique (45 minutes)

- **Objectif** : Comprendre l'écosystème Qt et configurer l'environnement de développement.
- **Contenu** :
 - Widgets de base : QWidget, QPushButton, QLabel, QLineEdit, QCheckBox
 - Layouts : QHBoxLayout, QVBoxLayout, QGridLayout
 - Personnalisation de fenêtre

Introduction à Qt

Qu'est-ce que Qt ?



- Qt est une **bibliothèque logicielle multiplateforme** (Windows, Linux, macOS, Android, iOS) utilisée principalement pour développer des **interfaces graphiques (GUI)** et des **applications complètes**.
- Qt est une **bibliothèque orientée objet (API)** développée en **C++**.
- Qt fournit un ensemble de **classes** décrivant des éléments graphiques (**widgets**) et des éléments non graphiques : **accès aux données** (fichier, base de données), **connexions réseaux** (socket), **gestion du multitâche** (thread), **OpenGL**, etc.

Introduction à Qt

Pourquoi utiliser Qt ?



- **Productivité** : Qt **simplifie le développement** d'applications complexes grâce à ses outils et modules.
- **Portabilité**: Cela permet aux développeurs de **créer une application une fois** et de la **déployer sur plusieurs plateformes** avec peu ou pas de modifications.
- **Performance** : Qt est optimisé pour être **rapide** et **efficace**.
- **Communauté active** : Une **grande communauté** et une documentation complète.

Introduction à Qt

Que permet Qt ?



- Qt est particulièrement connu pour sa capacité à créer des **interfaces graphiques** riches et interactives.
- Il fournit des **widgets** (composants d'interface utilisateur) prêts à l'emploi, tels que :
 - Boutons (**QPushButton**)
 - Étiquettes (**QLabel**)
 - Champs de texte (**QLineEdit**)
 - Fenêtres (**QMainWindow**)
 - Et bien d'autres.
- Ces widgets sont **personnalisables** et peuvent être disposés à l'aide de **layouts** (gestionnaires de placement) pour créer des interfaces complexes.

Introduction à Qt

Modules et fonctionnalités



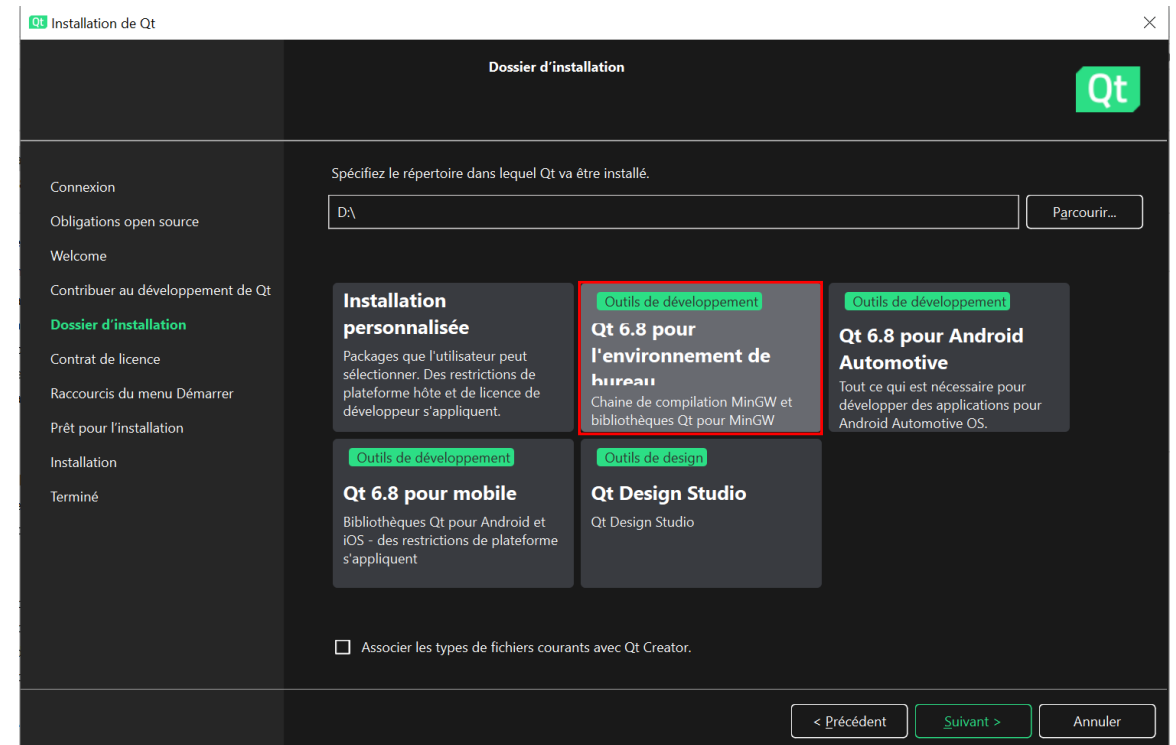
Qt est bien plus qu'une simple bibliothèque pour les interfaces graphiques. Il est composé de **modules** qui couvrent un large éventail de fonctionnalités :

- **Qt Core** : Fonctionnalités de base (gestion des événements, fichiers, threads, etc.).
- **Qt GUI** : Dessin et gestion des fenêtres.
- **Qt Widgets** : Widgets pour créer des interfaces graphiques.
- **Qt Network** : Gestion des communications réseau.
- **Qt SQL** : Accès aux bases de données.
- **Qt Multimedia** : Gestion de l'audio et de la vidéo.
- **Qt WebEngine** : Intégration de navigateurs web.
- **Qt Quick** : Framework pour créer des interfaces modernes et dynamiques avec QML (un langage déclaratif).

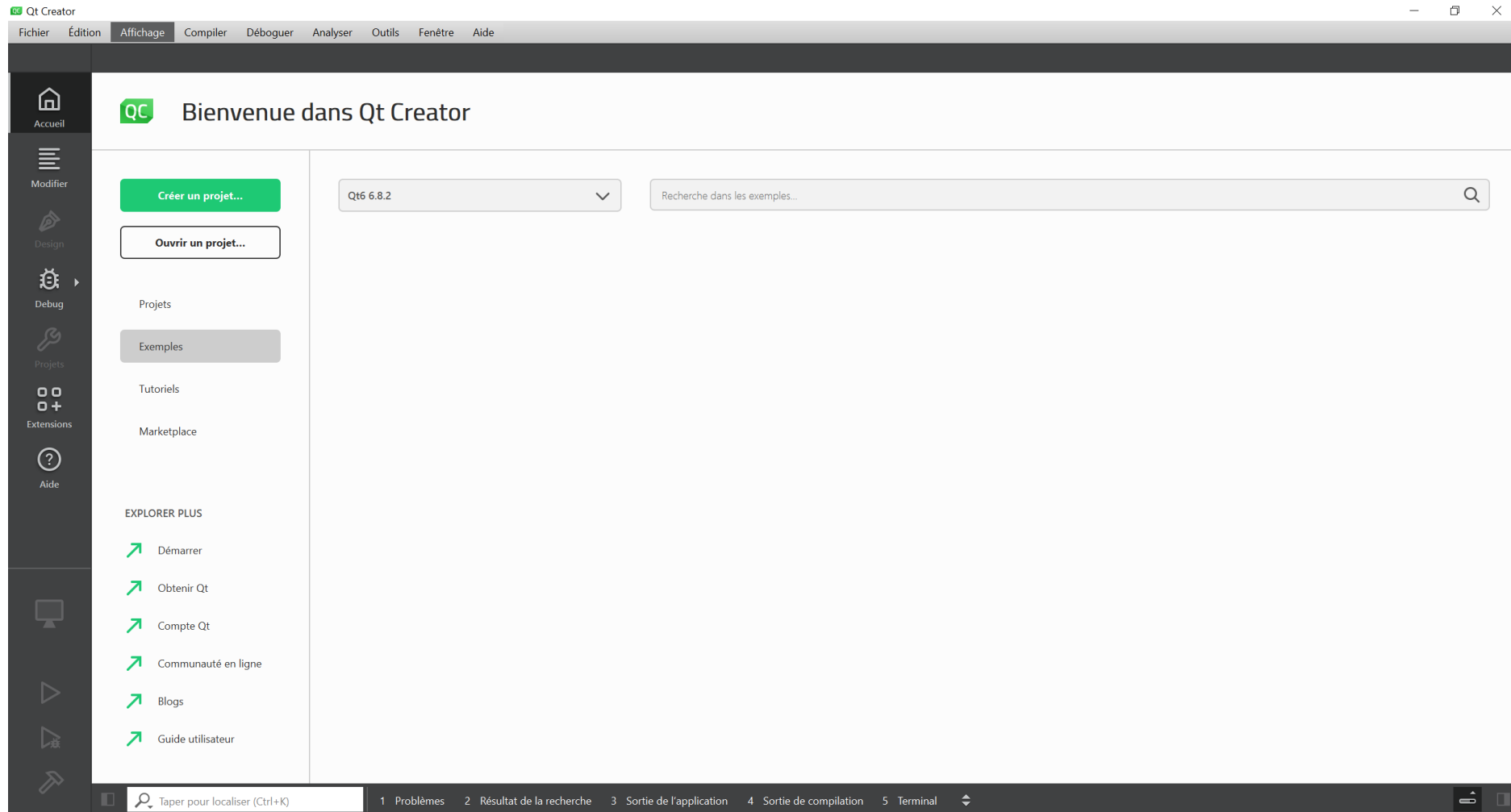
Installation de Qt

- **Étapes :**

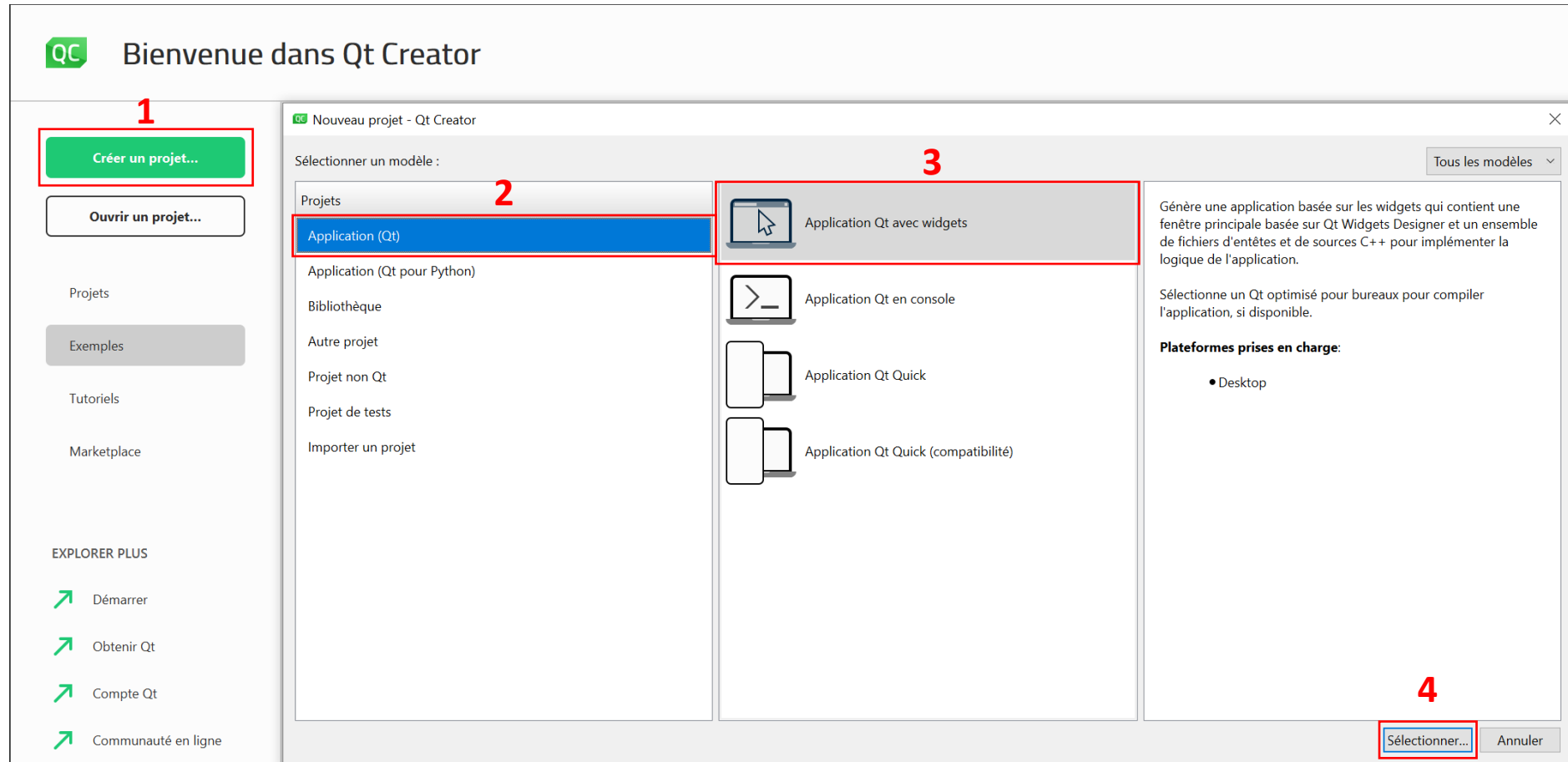
1. Télécharger Qt depuis qt.io.
2. Installer Qt Creator (IDE intégré).
3. Configurer un kit de compilation (MinGW/MSVC pour Windows, GCC pour Linux).



QT Creator



Première application



Première application

 Application Qt avec widgets 

Emplacement

Système de compilation

▷ Détails

Traduction

Kits

Résumé

Information sur la classe

Définir les informations de base des classes pour lesquelles vous souhaitez générer des fichiers squelettes de code source.

Nom de la classe :

FirstWidget

Classe parent :

QWidget

Fichier d'en-tête :

firstwidget.h

Fichier source :

firstwidget.cpp

☒ Générer le formulaire

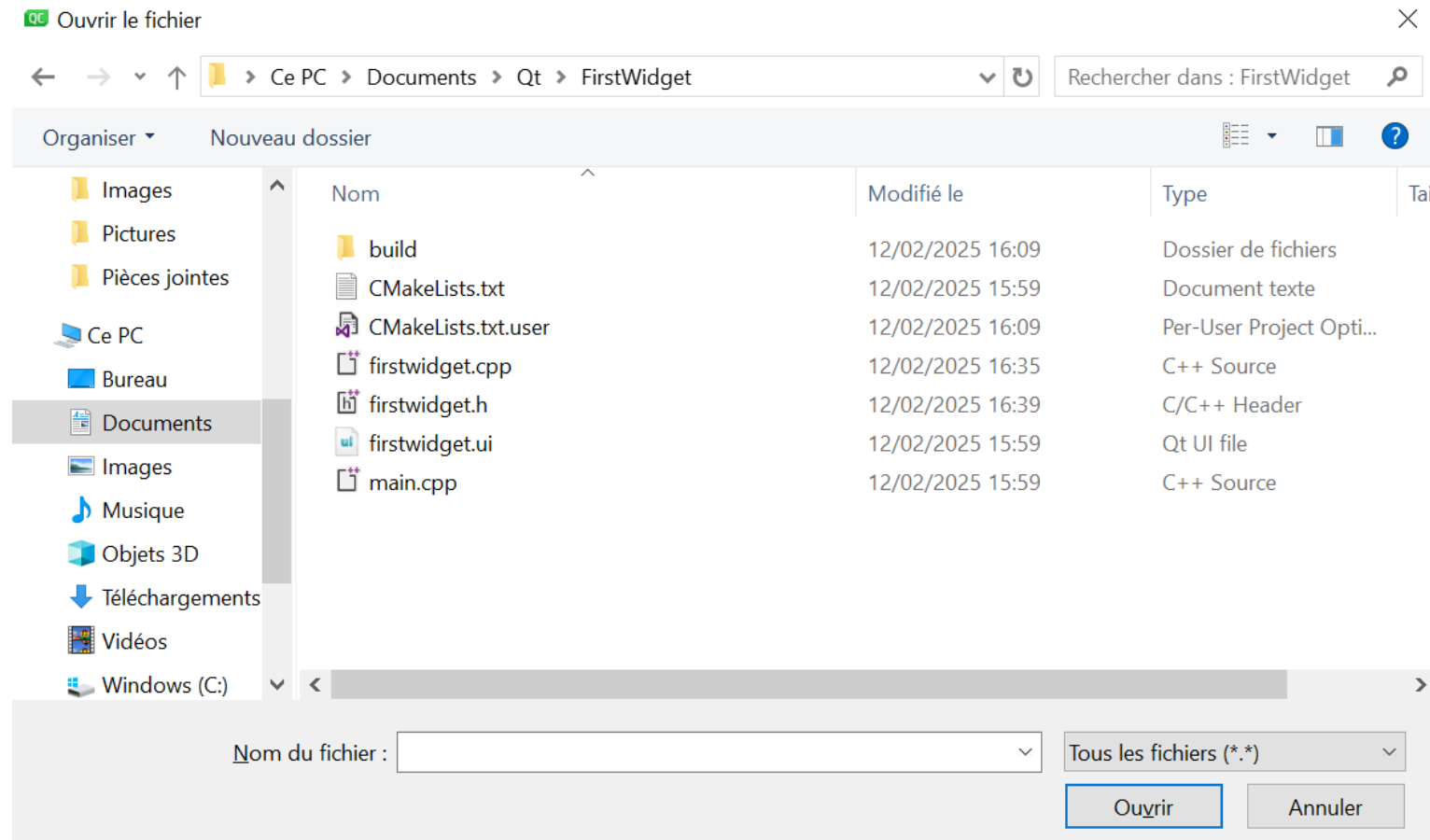
Fichier d'interface :

firstwidget.ui

Suivant >

Annuler

Première application



Première application

firstwidget.h

firstwidget.h @ FirstWidget - Qt Creator

Fichier Édition Affichage Compiler Déboguer Analyser Outils Fenêtre Aide

Projets

FirstWidget

- CMakelists.txt
- FirstWidget
 - CMakelists.txt:23
 - Forms
 - firstwidget.ui
 - Header Files
 - firstwidget.h
 - Source Files
 - firstwidget.cpp
 - main.cpp

```
1 #include <QMainWindow>
2
3
4 class FirstWidget : public QMainWindow
5 {
6
7 public:
8
9     FirstWidget(QWidget *parent = nullptr);
10
11     ~FirstWidget();
12
13 };
14
15
```

Inclusion de la classe QMainWindow pour créer des fenêtres principales dans l'application

Déclaration de la classe FirstWidget qui hérite de QMainWindow (Qui lui-même hérite de la classe QWidget)

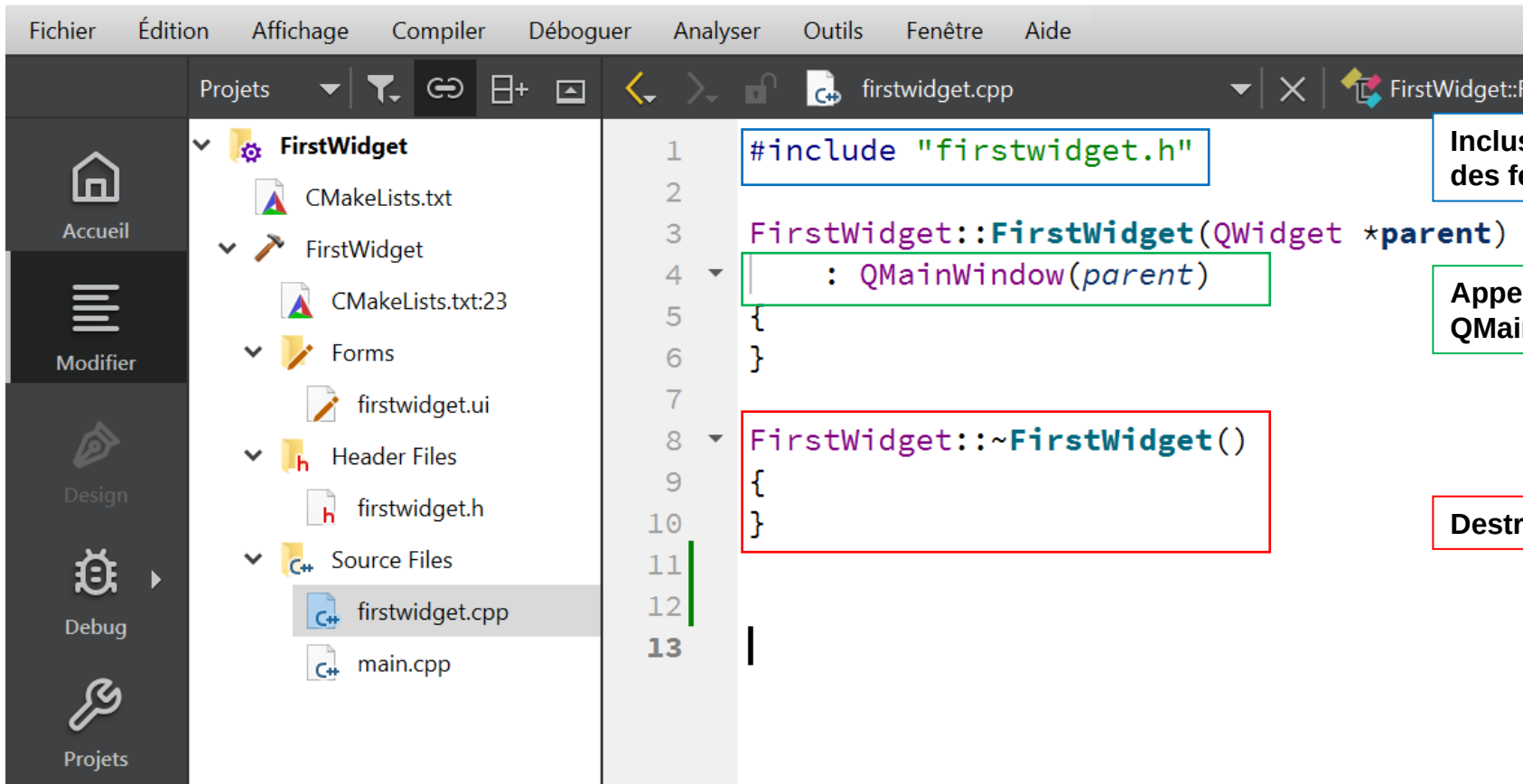
Constructeur de la classe FirstWidget avec un paramètre parent par défaut à nullptr

Destructeur de la classe FirstWidget

Première application

firstwidget.cpp

firstwidget.cpp @ FirstWidget - Qt Creator



```
1 #include "firstwidget.h"
2
3 FirstWidget::FirstWidget(QWidget *parent)
4     : QMainWindow(parent)
5 {
6 }
7
8 FirstWidget::~FirstWidget()
9 {
10 }
11
12
13
```

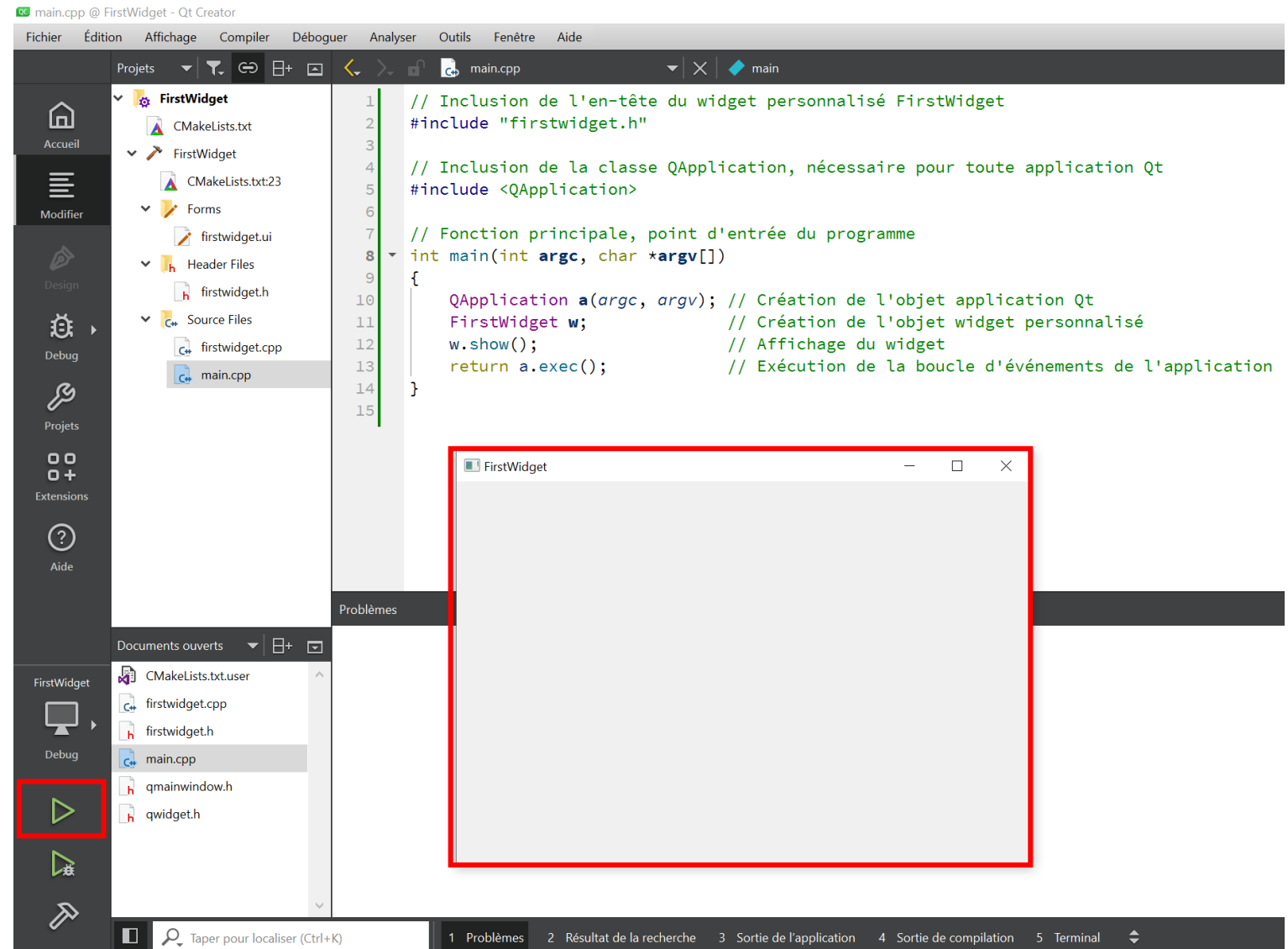
Inclusion de la classe QMainWindow pour créer des fenêtres principales dans l'application

Appel du constructeur de la classe parente QMainWindow avec le parent spécifié

Destructeur de la classe FirstWidget

Première application

main.cpp



Application GUI

La classe QApplication

Pour créer une application Qt graphique (GUI), il faut instancier un objet de la classe **QApplication** :

- **QApplication** est la classe de base qui **gère le cycle de vie** d'une application Qt.
- **QApplication** fournit une **boucle principale d'événements** pour les applications Qt : tous les événements du système seront traités et expédiés.
- C'est sa méthode `exec()` qui exécute la boucle d'attente des événements (clics de souris, frappes au clavier, redimensionnements de fenêtre,...) jusqu'à la fermeture du dernier objet de l'application.
- Elle attend en permanence que des événements se produisent, les traite, et les transmet aux objets (**widgets**) concernés.
- La classe **QApplication** gère également l'initialisation et la finalisation de l'application, ainsi que ses paramètres.
- L'instance de **QApplication** doit être créée avant tout objet graphique.

Application GUI

La classe QApplication

```
#include <QApplication>

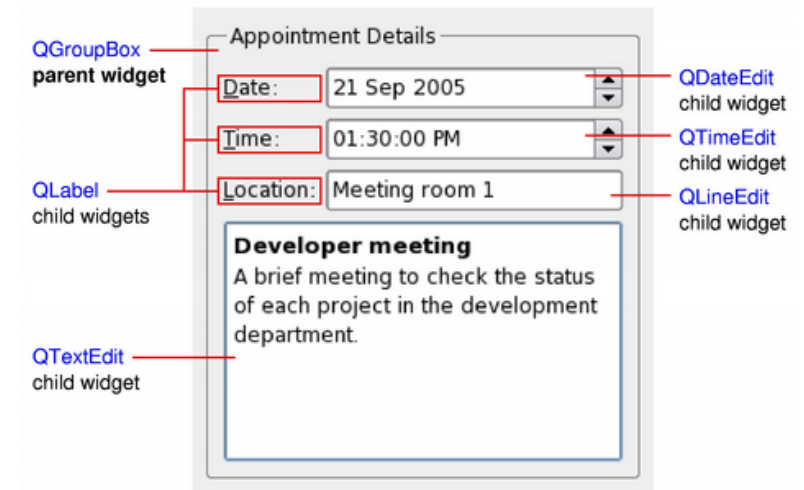
int main(int argc, char **argv)
{
    // Création d'un objet QApplication
    QApplication app(argc, argv);

    return app.exec();
}
```

Application GUI

La classe QWidget

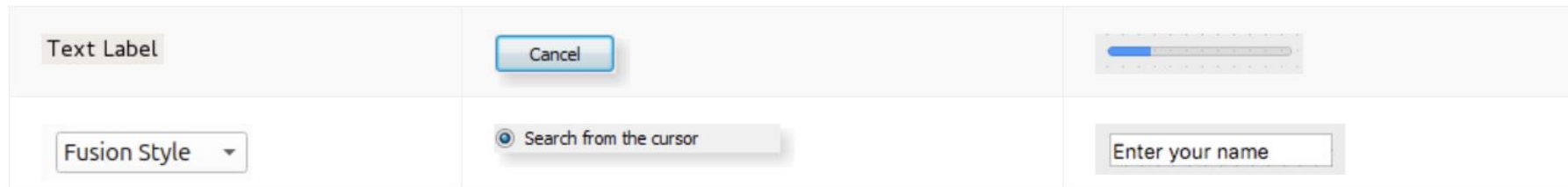
- Avec Qt, les éléments graphiques prédéfinis sont appelés des **widgets** (pour windows gadgets).
- Les **widgets** sont des éléments d'interface graphique (GUI) utilisés pour construire des applications interactives.
- les **widgets** sont des composants visuels qui permettent à l'utilisateur d'interagir avec l'application.
- Ils peuvent inclure des boutons, des zones de texte, des menus, des fenêtres, etc.



Application GUI

La classe QWidget

- La classe **QWidget** fournit la capacité de base d'affichage à l'écran et de gestion des événements.
- Elle est la **classe mère** de toutes les classes servant à réaliser des interfaces graphiques.
- Tous les widgets héritent de la classe de base **QWidget**
- Cela signifie qu'ils partagent des fonctionnalités communes, comme la gestion des événements, la mise en page, et le rendu à l'écran.
- **Widgets de base** : *QPushButton*, *QLabel*, *QLineEdit*, *QCheckBox*, etc.

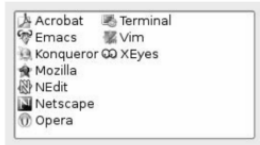


- Consulter toutes les classes [ici](#)

Application GUI

Les widgets prédéfinis

Qt fournit des **widgets prédéfinis** permettant de composer ses propres applications graphiques :



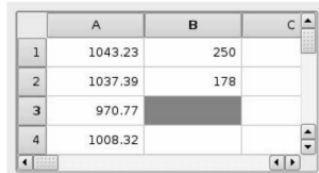
QListView(liste)



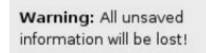
QListView(icônes)



QTreeView



QTableView



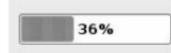
QLabel(texte)



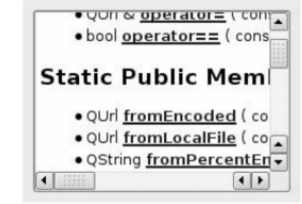
QLabel (image)



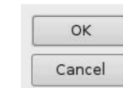
QLCDNumber



QProgressBar



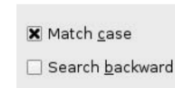
QTextBrowser



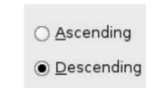
QPushButton



QToolButton



QCheckBox



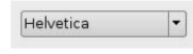
QRadioButton



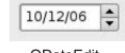
QSpinBox



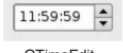
QDoubleSpinBox



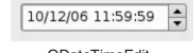
QComboBox



QDateEdit



QTimeEdit



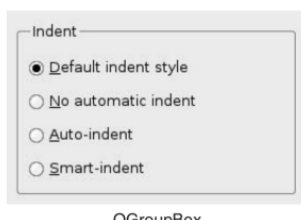
QDateTimeEdit



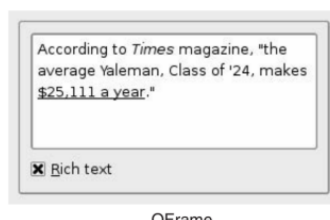
QScrollBar



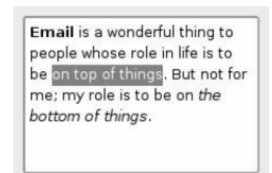
QSlider



QGroupBox



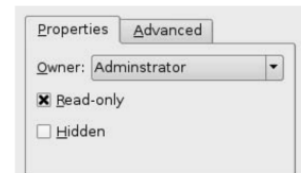
QFrame



QTextEdit



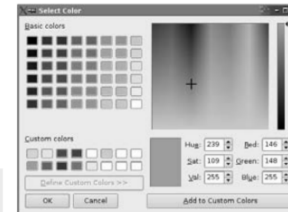
QLineEdit



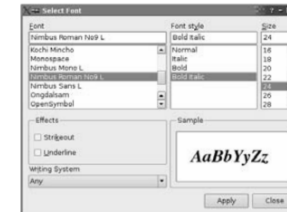
QTabWidget



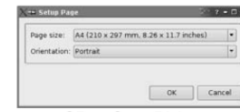
QToolBox



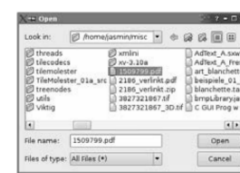
QColorDialog



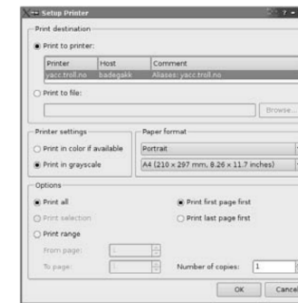
QFontDialog



QPageSetupDialog



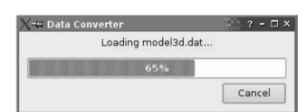
QFileDialog



QPrintDialog



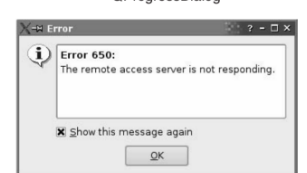
QInputDialog



QProgressDialog



QMessageBox



QErrorMessage

Application GUI

La classe QApplication

```
#include <QApplication>
#include <QWidget>
#include <QPushButton>

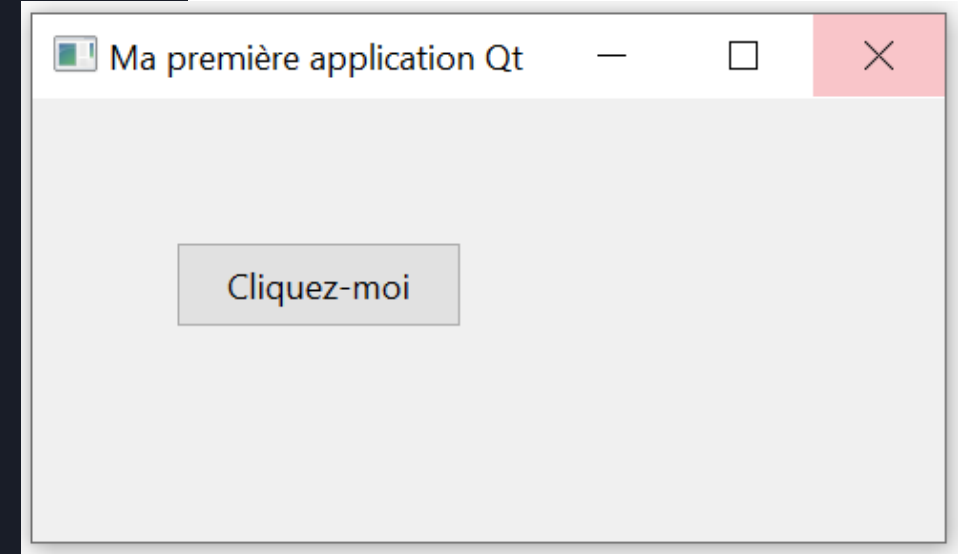
int main(int argc, char *argv[]) {
    // 1. Création de l'instance QApplication
    QApplication app(argc, argv);

    // 2. Création d'une fenêtre principale
    QWidget window;
    window.setWindowTitle("Ma première application Qt");

    // 3. Ajout d'un bouton à la fenêtre
    QPushButton button("Cliquez-moi", &window);
    button.setGeometry(50, 50, 100, 30); // Position et taille du bouton

    // 4. Affichage de la fenêtre
    window.show();

    // 5. Démarrage de la boucle d'événements
    return app.exec();
}
```



Application GUI

Propriétés des widgets (1/3)

- Les **widgets** dans Qt ont de nombreuses propriétés qui permettent de personnaliser leur apparence et leur comportement.
- **QTextEdit** : est un widget qui permet d'afficher et d'éditer du texte formaté

- Définir le texte :

```
QTextEdit textEdit;  
textEdit.setText("Ceci est un QTextEdit."); // Définit le texte
```

- Définir la police :

```
textEdit.setFont(QFont("Arial", 12)); // Définit la police et la taille
```

- Définir la couleur de fond :

```
textEdit.setStyleSheet("background-color: lightyellow;"); // Couleur de fond
```

- Activer/désactiver l'édition :

```
textEdit.setReadOnly(true); // Rend le QTextEdit en lecture seule
```

Application GUI

Propriétés des widgets (2/3)

- **QPushButton** : est un bouton cliquable qui permet à l'utilisateur de déclencher des actions.

- Définir le texte du bouton :

```
QPushButton button("Cliquez-moi !"); // Texte initial  
button.setText("Nouveau Texte"); // Modifie le texte
```

- Définir la taille :

```
button.resize(100, 50); // Définit la taille du bouton
```

- Définir la police :

```
button.setFont(QFont("Arial", 14, QFont::Bold)); // Police personnalisée
```

- Activer/désactiver le bouton :

```
button.setEnabled(false); // Désactive le bouton
```

Application GUI

Propriétés des widgets (3/3)

- **QMessageBox** : est une boîte de dialogue utilisée pour afficher des messages à l'utilisateur (informations, avertissements, erreurs, etc.).
 - **Personnaliser les boutons :**

```
QMessageBox msgBox;  
msgBox.setText("Voulez-vous enregistrer les modifications ?");  
msgBox.setStandardButtons(QMessageBox::Save | QMessageBox::Discard | QMessageBox::Cancel);  
msgBox.setDefaultButton(QMessageBox::Save);  
int ret = msgBox.exec(); // Affiche la boîte de dialogue et attend la réponse
```

... et bien d'autres propriétés à voir dans les TPs

Gestionnaire de placement

Positionnement des widgets

- Vous pouvez gérer le placement vos widgets de deux manières :
 - avec un **positionnement absolu** (généralement déconseillé car il pose des problèmes de résolution d'écran, de redimensionnement, ...)
 - avec un **positionnement relatif**

```
#include <QApplication>
#include <QWidget>
#include <QPushButton>

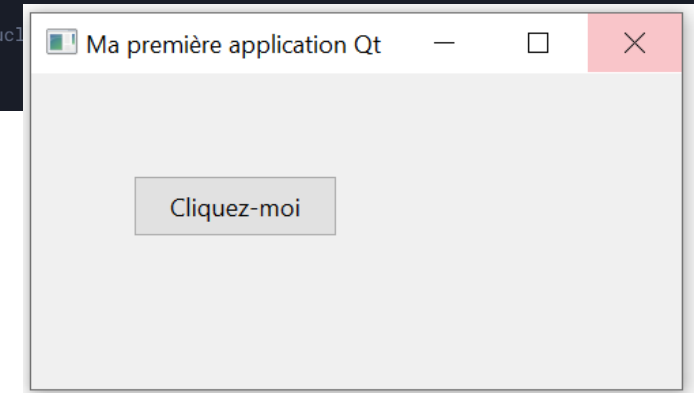
int main(int argc, char *argv[]) {
    // 1. Création de l'instance QApplication
    QApplication app(argc, argv);

    // 2. Création d'une fenêtre principale
    QWidget window;
    window.setWindowTitle("Ma première application Qt");

    // 3. Ajout d'un bouton à la fenêtre
    QPushButton button("Cliquez-moi", &window);
    button.setGeometry(50, 50, 100, 30); // Position et taille du bouton

    // 4. Affichage de la fenêtre
    window.show();

    // 5. Démarrage de la boucle
    return app.exec();
}
```



Gestionnaire de placement

Les layouts

- Les **layouts** (mises en page) dans Qt sont des outils puissants pour **organiser** et **gérer** la disposition des widgets dans une interface graphique.
- Ils permettent de positionner les widgets de manière **dynamique** et **flexible**, en s'adaptant à la **taille de la fenêtre ou du conteneur (positionnement relatif)**.
- Cela **évite** de devoir définir **manuellement** les **positions** et les **tailles** des widgets, ce qui rend l'interface **plus robuste** et **plus facile à maintenir**.

Gestionnaire de placement

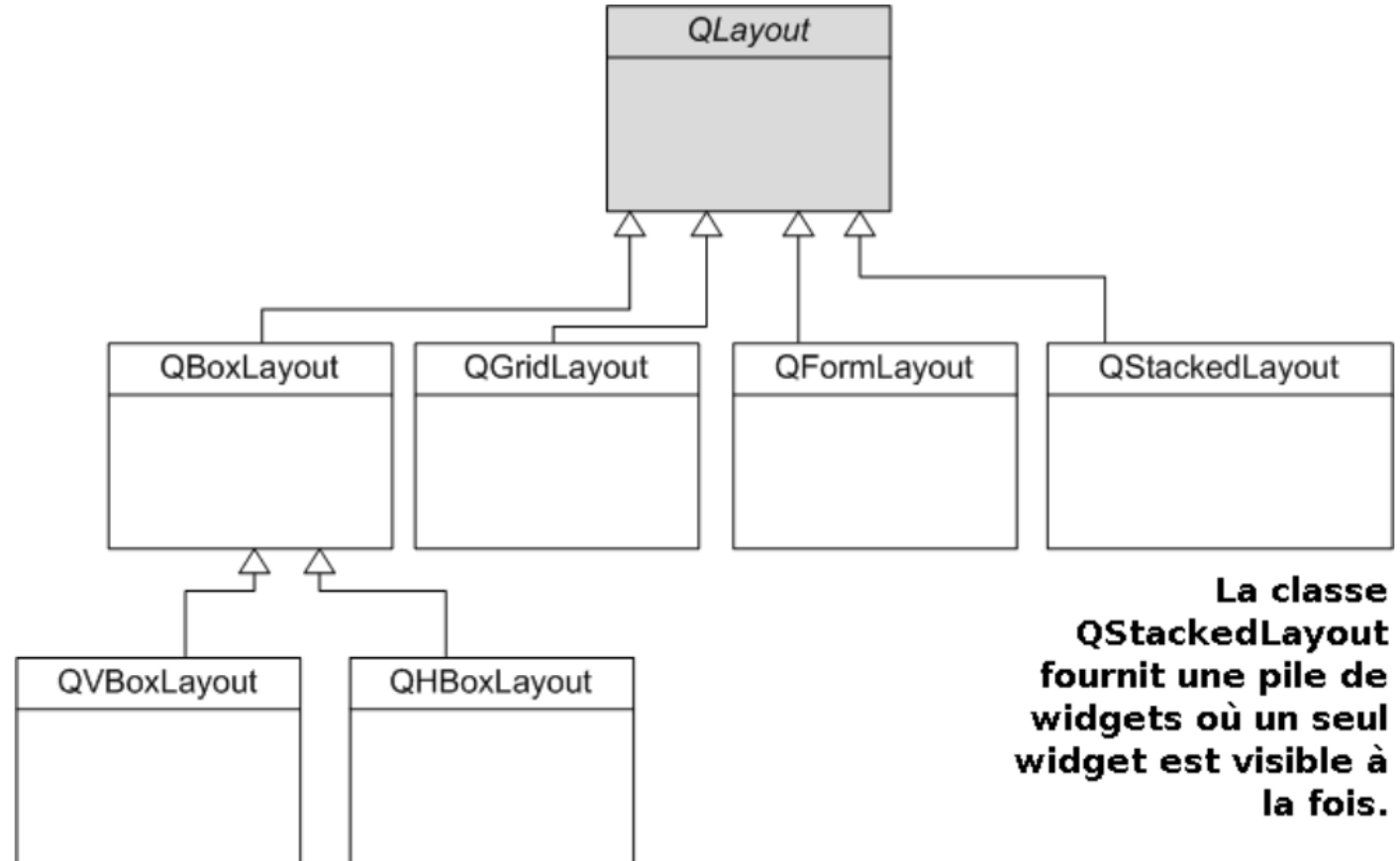
Pourquoi utiliser des layouts ?

- **Adaptabilité** : Les layouts **ajustent automatiquement** la **taille** et la **position** des widgets lorsque la fenêtre est redimensionnée.
- **Organisation** : Ils permettent de structurer l'interface de manière logique (par exemple, en **lignes**, **colonnes**, **grilles**, etc.).

Gestionnaire de placement

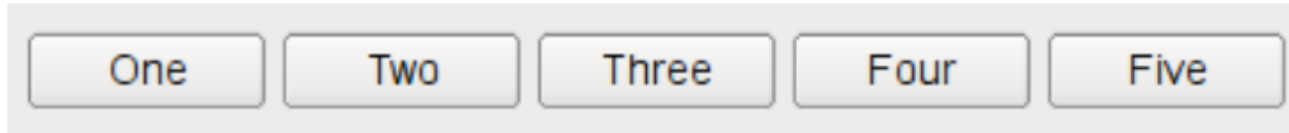
Les classes QLayout (classe abstraite)

Qt inclut un ensemble de classes **QxxxLayout** qui sont utilisés pour décrire la façon dont les widgets sont disposés dans l'interface utilisateur d'une application.



Gestionnaire de placement

Les classes QLayout

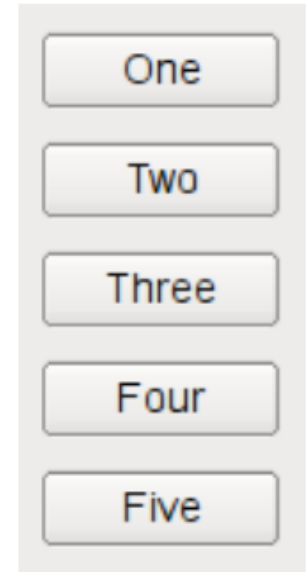
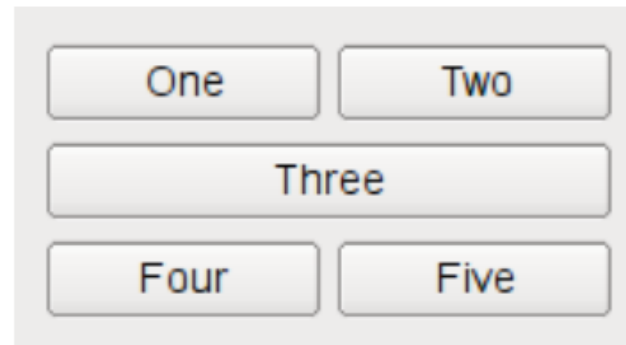


QHBoxLayout

QFormLayout



QGridLayout



QVBoxLayout

Gestionnaire de placement

Principe

- Les gestionnaires de disposition (les classes **QxxxLayout**) simplifient le travail de positionnement :
 - on peut ajouter des widgets dans un layout :
`QLayout::addWidget(QWidget *widget)`
 - on peut ajouter des layouts dans un layout :
`QLayout::addWidget(QLayout *layout)`
 - on peut associer un layout à un widget qui devient alors le propriétaire du layout et parent des widgets inclus dans le layout :
`QWidget::setLayout (QLayout *layout)`

Gestionnaire de placement

Exemple : Plusieurs boutons, QHBoxLayout et QVBoxLayout

Placement directement dans la fenêtre

```
QWidget window;  
  
QPushButton *b1 = new QPushButton("1", &window);  
QPushButton *b2 = new QPushButton("2", &window);  
QPushButton *b3 = new QPushButton("3", &window);
```

Placement dans un layout

```
QWidget window;  
  
QPushButton *b1 = new QPushButton("1");  
QPushButton *b2 = new QPushButton("2");  
QPushButton *b3 = new QPushButton("3");  
  
QHBoxLayout *layout = new QHBoxLayout;  
layout->addWidget(b1);  
layout->addWidget(b2);  
layout->addWidget(b3);  
  
// c'est le layout qui est placé dans la fenêtre  
window.setLayout(layout);
```

Gestionnaire de placement

Exemple : Plusieurs boutons, QGridLayout

Organise les widgets dans une grille (lignes et colonnes)

```
QWidget window;  
  
QPushButton *b1 = new QPushButton("1");  
QPushButton *b2 = new QPushButton("2");  
QPushButton *b3 = new QPushButton("3");  
  
QGridLayout *layout = new QGridLayout;  
layout->addWidget(b1, 0, 0);  
layout->addWidget(b2, 0, 1);  
layout->addWidget(b3, 1, 0, 1, 2); // Ligne 1, Colonne 0, occupe 1 ligne et 2 colonnes  
  
window.setLayout(layout); // c'est le layout qui est placé dans la fenêtre
```

Gestionnaire de placement

Exemple : QFormLayout

Organise les widgets sous forme de **formulaire** (étiquettes et champs)

```
QWidget window;
```

```
QFormLayout *layout = new QFormLayout;
```

```
layout->addRow("Nom :", new QLineEdit);
```

```
layout->addRow("Âge :", new QLineEdit);
```

```
layout->addRow("Adresse :", new QLineEdit);
```

```
window.setLayout(layout);
```

Gestionnaire de placement

Exemple : Combinaison de layouts

- Les **layouts** peuvent être **imbriqués** pour créer des interfaces complexes.
- Par exemple, un **QVBoxLayout** peut contenir un **QHBoxLayout**, qui lui-même contient des widgets.

```
QVBoxLayout *mainLayout = new QVBoxLayout;
```

```
QHBoxLayout *topLayout = new QHBoxLayout;
```

```
topLayout->addWidget(new QPushButton("Gauche"));
```

```
topLayout->addWidget(new QPushButton("Droite"));
```

```
QHBoxLayout *bottomLayout = new QHBoxLayout;
```

```
bottomLayout->addWidget(new QPushButton("Bas 1"));
```

```
bottomLayout->addWidget(new QPushButton("Bas 2"));
```

```
mainLayout->addLayout(topLayout);
```

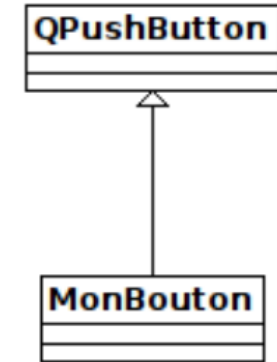
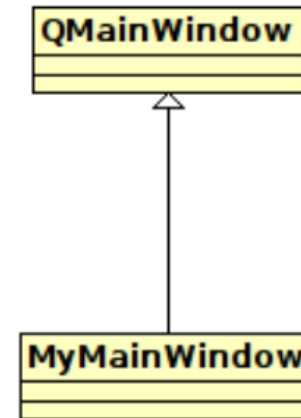
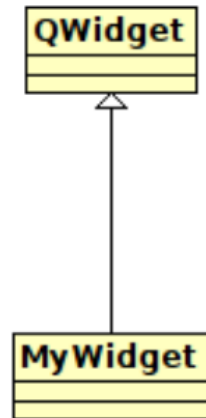
```
mainLayout->addLayout(bottomLayout);
```

```
window.setLayout(mainLayout);
```

Personnalisation

Création de widget et de fenêtre personnalisées

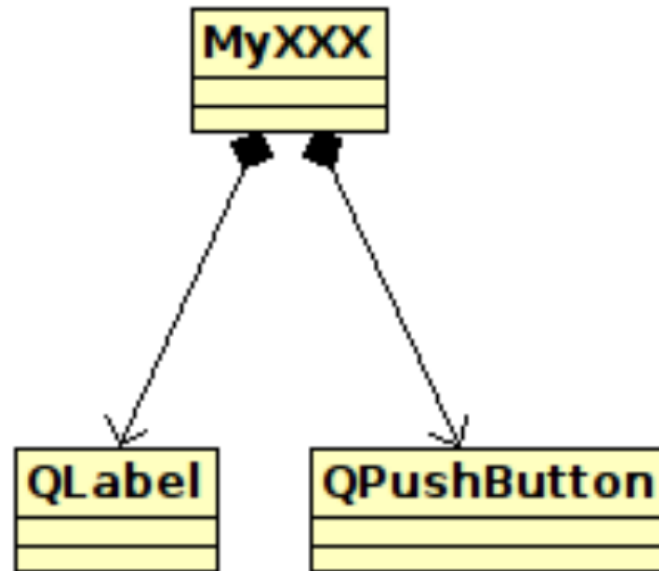
- La création de **widgets** personnalisés est faite en héritant de **QWidget** ou une **classe fille**
- La création de **fenêtres** personnalisées est faite en héritant de **QWidget**, **QDialog** ou **QMainWindow** :



Personnalisation

Création d'une fenêtre personnalisée

- Ensuite, on compose sa fenêtre personnalisée en y intégrant des widgets :



Personnalisation

Création d'une fenêtre personnalisée en héritant de QWidget

Création d'une nouvelle classe qui hérite de la classe QWidget (**MyFirstClass.h** et **MyFirstClass.cpp**)

1. Fichier d'en-tête (**MyFirstClass.h**)

```
#ifndef DEF_MYFIRSTAPPLICATION // Pour éviter les problèmes liés aux « include » plusieurs fois
#define DEF_MYFIRSTAPPLICATION
#include <QWidget>
#include <QPushButton>
#include <QLabel>

class MyFirstClass : public QWidget {
public:
    MyFirstClass(); // Constructeur
private:
    QPushButton *m_bouton; // Un bouton comme membre privé
    QLabel *m_label;       // Un label comme membre privé
};
#endif
```

Personnalisation

Création d'une fenêtre personnalisée en héritant de **QWidget**

2. Fichier d'implémentation (**MyFirstClass.cpp**)

```
#include "MyFirstClass.h"
```

```
MyFirstClass::MyFirstClass() : QWidget()
```

```
{
```

```
    m_bouton = new QPushButton("Cliquez-moi !", this); // Création du bouton
    m_bouton->setFont(QFont("Comic Sans MS", 14)); // Personnalisation du bouton
    m_bouton->setCursor(Qt::PointingHandCursor);
```

```
    m_label = new QLabel("Ceci est un QLabel", this); // Création du label
    m_label->setFont(QFont("Arial", 12)); // Personnalisation du label
    m_label->setAlignment(Qt::AlignCenter); // Centrer le texte
```

```
    QVBoxLayout *layout = new QVBoxLayout(this); // Organisation avec un layout
    layout->addWidget(m_label); // Ajouter le label en premier
    layout->addWidget(m_bouton); // Ajouter le bouton en dessous
    setLayout(layout); // Définir le layout pour le widget
```

```
}
```

Personnalisation

Création d'une fenêtre personnalisée en héritant de QWidget

3. Fichier de la fonction principale (main.cpp)

```
#include <QApplication>
#include "MyFirstClass.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    // Création d'une instance du widget personnalisé
    MyFirstClass myFCClass;

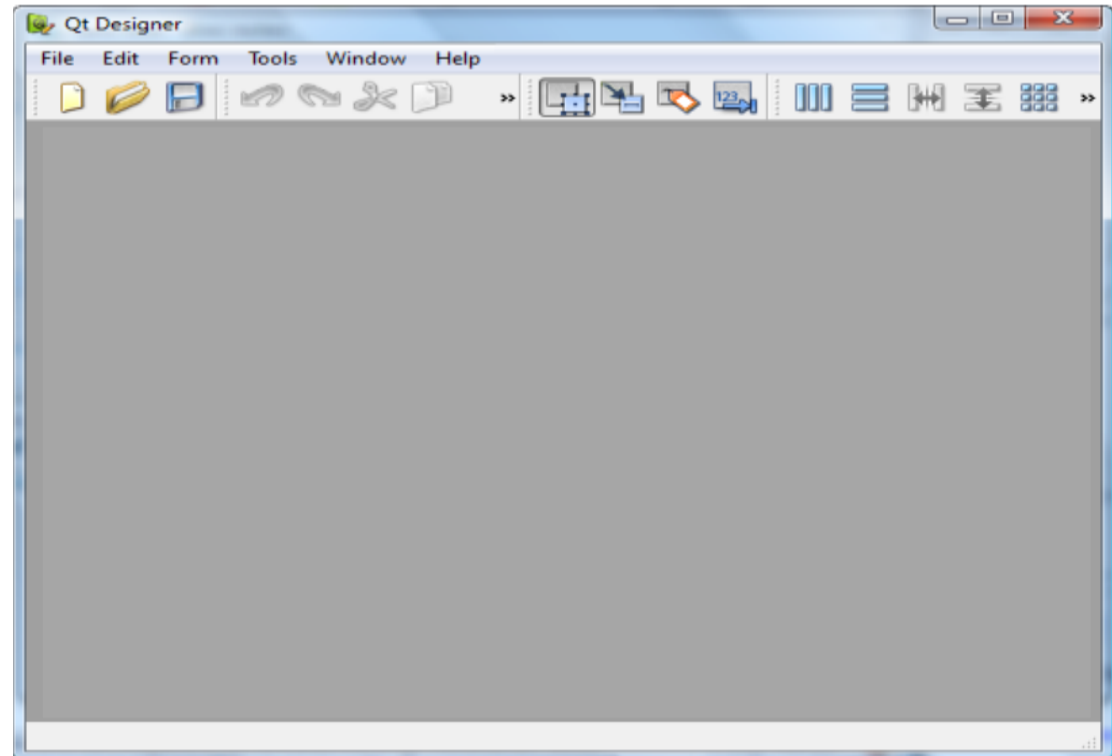
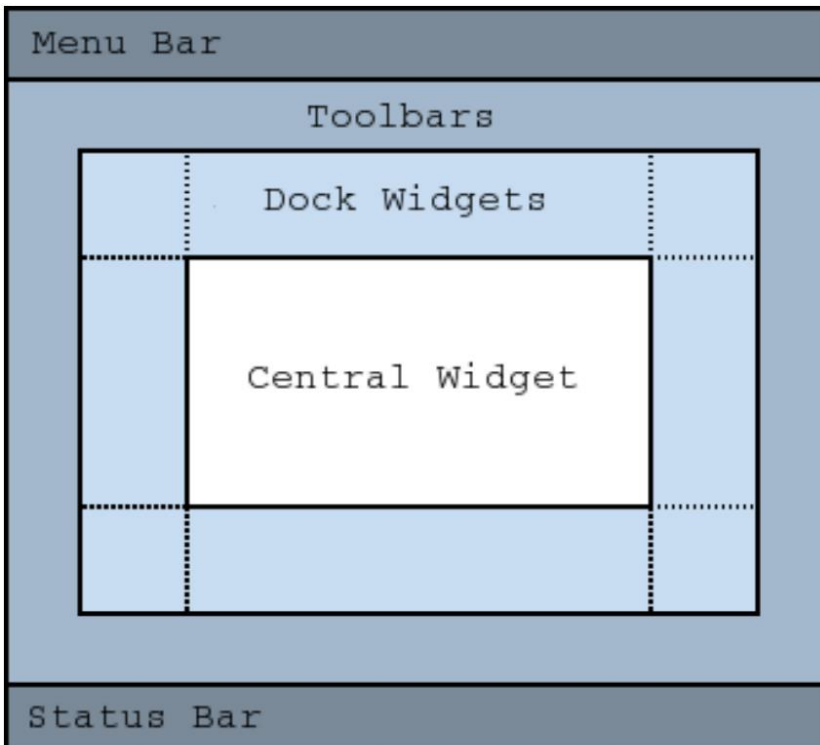
    myFCClass.show();

    return app.exec();
}
```

Personnalisation

Création d'une fenêtre personnalisée en héritant de QMainWindow

La classe `QMainWindow` propose un « squelette » permettant de créer une fenêtre principale avec une **barre de menus**, une **barre d'outils**, une **barre d'état**, etc.



Personnalisation

Création d'une fenêtre personnalisée en héritant de QMainWindow

1. Héritage de QMainWindow

```
#include <QMainWindow>

class MyFirstClass : public QMainWindow {
public:
    MyFirstClass(); // Constructeur
private:
    // Ici, vous pouvez ajouter des membres privés (comme des menus, des
    boutons, etc.)
};
```

Personnalisation

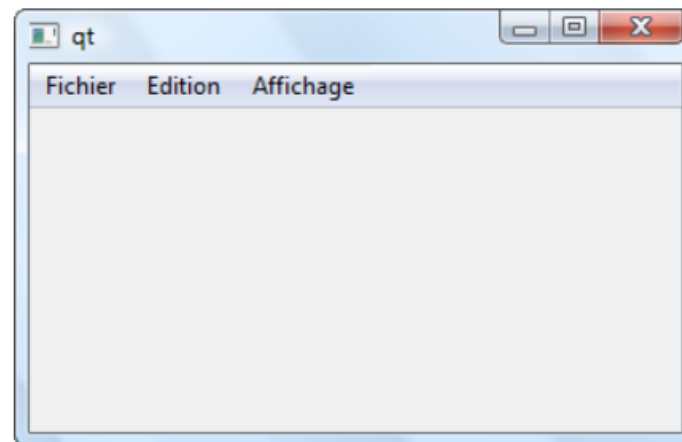
Création d'une fenêtre personnalisée en héritant de QMainWindow

2. Définition des menus

```
#include <QMainWindow>
#include <QMenu>

class MyFirstClass :
    public QMainWindow
{
    public:
        // Constructeur
        MyFirstClass();
    private:
        // Menu
        QMenu *mFichier;
        QMenu *mEdition;
        QMenu *mAffichage;
};
```

```
MyFirstClass::MyFirstClass() :QMainWindow()
{
    // Création des menus
    mFichier = menuBar()->addMenu("&Fichier");
    mEdition = menuBar()->addMenu("&Edition");
    mAffichage = menuBar()->addMenu("&Affichage");
}
```



Personnalisation

Création d'une fenêtre personnalisée en héritant de QMainWindow

3. Ajouter des actions aux menus

Un menu sans **actions** ne sert à rien. Tu peux ajouter des actions (comme "Ouvrir", "Enregistrer", "Quitter", etc.) à chaque menu en utilisant la classe QAction.

```
MyFirstClass::MyFirstClass() :QMainWindow()  
{  
    // Création des menus  
    mFichier = menuBar()->addMenu("&Fichier");  
    mEdition = menuBar()->addMenu("&Edition");  
    mAffichage = menuBar()->addMenu("&Affichage");  
  
    // Ajouter des actions au menu Fichier  
    QAction *actionQuitter = new QAction("&Quitter", this);  
    mFichier->addAction(actionQuitter);  
}
```

